

DATA STRUCTURES & ALGORITHMS

The Complete Self-Contained Study Guide

KR Mangalam University | B.Tech CSE | ETCCDS202 | 4 Credits

Everything you need — no internet, no other books required

□ WHAT'S INSIDE THIS GUIDE

UNIT 1 → Foundations: Big O, Complexity, Recursion, Algorithm Paradigms

UNIT 2 → Linear Structures: Arrays, Linked Lists, Stacks, Queues

UNIT 3 → Sorting Algorithms: InsertionSort, MergeSort, QuickSort + Performance

UNIT 4 → Non-Linear: BST, Heaps, Graphs, Hashing, Tries, Balanced Trees

LAB 1 → Recursive Problem Solving: Factorial, Fibonacci, Hanoi, Binary Search

LAB 2 → Linear DS Implementation: LinkedList, Stack, Queue, Parentheses Checker

LAB 3 → Sorting Performance Measurement & Comparison

LAB 4 → BST, Graph Traversal, HashTable with Collision Handling

LAB 5 → Capstone: Social Network Explorer (full implementation)

EXTRAS → Viva Q&A, Coding Questions, Exam Tips, Complexity Cheat Sheet

UNIT 1 — Foundations & Algorithmic Analysis (10 hrs)

1.1 What is a Data Structure?

A data structure is a way of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently. Think of it like different types of containers — a stack of plates, a queue at a counter, a tree of folders.

Why do we need them?

- Raw data without structure is useless — you can't search, sort, or process it efficiently.
- The right data structure can turn an $O(n^2)$ algorithm into $O(\log n)$ — the difference between seconds and years for large data.
- Real-world systems (databases, operating systems, browsers, GPS) are built entirely on data structures.

1.2 What is an Algorithm?

An algorithm is a step-by-step procedure to solve a problem. It must be:

- **Finite:** Must terminate after a finite number of steps.
- **Definite:** Each step must be clear and unambiguous.
- **Input/Output:** Takes 0 or more inputs, produces at least 1 output.
- **Effective:** Each step must be feasible (doable by a computer).

1.3 Time & Space Complexity — Big O Notation

Big O notation describes HOW AN ALGORITHM'S RESOURCE USAGE GROWS as the input size n increases. It ignores constants and lower-order terms — we care about the dominant growth rate.

The 7 Key Complexities — From Fastest to Slowest

$O(1)$	→ CONSTANT	→ Always same time regardless of input. e.g. array[5], hash lookup
$O(\log n)$	→ LOGARITHMIC	→ Halves problem each step. e.g. Binary search on 1M items = 20 steps
$O(n)$	→ LINEAR	→ One operation per item. e.g. Find max in unsorted list
$O(n \log n)$	→ LINEARITHMIC	→ n items, each taking $\log n$. e.g. MergeSort, QuickSort avg
$O(n^2)$	→ QUADRATIC	→ Nested loops over n items. e.g. BubbleSort, InsertionSort worst
$O(2^n)$	→ EXPONENTIAL	→ Doubles each step. e.g. Recursive Fibonacci, subset generation
$O(n!)$	→ FACTORIAL	→ e.g. Brute-force Travelling Salesman. Unusable beyond $n=15$

How to Calculate Big O — Rules

- **Drop constants:** $O(2n) \rightarrow O(n)$. We don't care if it's 2x or 10x, only growth rate.
- **Drop lower terms:** $O(n^2 + n) \rightarrow O(n^2)$. As $n \rightarrow \infty$, n^2 dominates completely.
- **Loops:** Single loop = $O(n)$. Nested loops = $O(n^2)$. Loop inside loop inside loop = $O(n^3)$.
- **Recursion:** Use Recurrence Relations. $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$ [Binary Search].
- **Sequence:** Two separate loops = $O(n) + O(n) = O(2n) = O(n)$. NOT $O(n^2)$.

Best, Average, and Worst Case

For the same algorithm, complexity can differ based on input:

```
# Linear Search Example:
# Best Case: O(1) → element is the first item
# Average Case: O(n/2) = O(n) → element is in the middle
# Worst Case: O(n) → element is last OR not present

# Why Worst Case matters most:
# It gives us the GUARANTEE — algorithm will NEVER be slower than this.
```

Space Complexity

Space complexity measures extra memory used (beyond the input itself).

- **O(1) auxiliary space:** InsertionSort, QuickSort (sorting in-place, only a few variables).
- **O(n) auxiliary space:** MergeSort (needs a copy array), Linked List (each node has extra pointer).
- **O(log n) stack space:** Recursive Binary Search, recursive QuickSort (recursion call stack depth).
- **O(n) stack space:** Recursive Fibonacci without memoization (depth of n).

1.4 Common Complexities Table

Algorithm	Best	Average	Worst	Space
Bubble Sort	O(n)	O(n ²)	O(n ²)	O(1) in-place
Selection Sort	O(n ²)	O(n ²)	O(n ²)	O(1) in-place
Insertion Sort	O(n)	O(n ²)	O(n ²)	O(1) in-place
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n) extra
Quick Sort	O(n log n)	O(n log n)	O(n ²)	O(log n) stack
Binary Search	O(1)	O(log n)	O(log n)	O(1)
BFS / DFS	O(V+E)	O(V+E)	O(V+E)	O(V)
BST insert/search	O(log n)	O(log n)	O(n) skewed	O(n)
Hash Table ops	O(1)	O(1)	O(n) collision	O(n)

1.5 Algorithm Design Paradigms

Divide & Conquer

Split the problem into smaller subproblems, solve each independently, then combine.

- Examples: MergeSort, QuickSort, Binary Search, Karatsuba multiplication
- Pattern: Divide → Conquer (recurse) → Combine

```
# MergeSort Divide & Conquer pattern:
# 1. DIVIDE: split array into left and right halves
# 2. CONQUER: recursively sort each half
# 3. COMBINE: merge the two sorted halves
```

Greedy

Make the locally optimal choice at each step, hoping it leads to a globally optimal solution. Fast but doesn't always give the best answer.

- Examples: Dijkstra's shortest path, Huffman coding, Activity Selection, Coin change (for standard coins)
- Key insight: Works when 'greedy choice property' holds — local best = global best

Dynamic Programming

Solve complex problems by breaking them into overlapping subproblems and storing results (memoization/tabulation) to avoid recomputation.

- Examples: Fibonacci (memoized), Knapsack, Longest Common Subsequence
- Key insight: If the same subproblem appears multiple times, store the result — $O(n)$ instead of $O(2^n)$

1.6 Recursion — Deep Dive

Recursion is when a function calls itself to solve a smaller version of the same problem. Every recursive solution needs:

- **Base Case:** The condition where the function STOPS calling itself. Without this → infinite recursion → StackOverflow.
- **Recursive Case:** The function calls itself with a SMALLER/Simpler input, moving toward the base case.
- **Stack Frames:** Each recursive call adds a frame to the call stack. Stack depth = space complexity.

How the Call Stack Works — Factorial(4) Traced

```
factorial(4)
└ 4 * factorial(3)
    └ 3 * factorial(2)
        └ 2 * factorial(1)
            └ returns 1          ← BASE CASE hit
                └ 2 * 1 = 2      ← unwind
                    └ 3 * 2 = 6
                        └ 4 * 6 = 24    ← final answer
```

Stack depth = 4 frames → Space: $O(n)$
Total multiplications = 4 → Time: $O(n)$

1.7 Lab Task 1 — Full Implementations

Factorial — Recursive

```
def factorial(n):
    # BASE CASE: factorial of 0 or 1 is 1
    if n == 0 or n == 1:
        return 1
    # RECURSIVE CASE: n * factorial of (n-1)
    return n * factorial(n - 1)

# Test:
print(factorial(0)) # 1
print(factorial(5)) # 120
print(factorial(10)) # 3628800

# Time Complexity: O(n)    — makes n recursive calls
# Space Complexity: O(n)   — n frames on call stack
```

Fibonacci — Recursive (Naive) vs Memoized

The naive recursive Fibonacci is the classic example of EXPONENTIAL time — and why memoization matters.

```
# — NAIVE RECURSIVE —————
def fib_naive(n):
    if n <= 0: return 0
    if n == 1: return 1
    return fib_naive(n-1) + fib_naive(n-2)
```

```

# Why  $O(2^n)$ ? Each call branches into 2 more calls:
#
#      /      \
#    fib(4)    fib(3)      ← fib(3) computed TWICE
#   /  \    /  \
# fib(3) fib(2) fib(2) fib(1) ← fib(2) computed 3 times!

# — MEMOIZED (top-down DP) —————
def fib_memo(n, memo={}):      # memo stores already computed values
    if n in memo: return memo[n] # Cache hit!  $O(1)$ 
    if n <= 1: return n
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]

# Time:  $O(n)$  — each value computed exactly once
# Space:  $O(n)$  — memo dict + call stack

# — ITERATIVE (bottom-up DP) —————
def fib_iter(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(2, n+1):
        a, b = b, a + b    # slide the window
    return b

# Time:  $O(n)$   Space:  $O(1)$  — best solution!

# Comparison for n=50:
# fib_naive:  $\sim 2^{50} = 1$  quadrillion operations (hours)
# fib_memo: 50 operations (microseconds)
# fib_iter: 50 operations, no stack overhead

```

Tower of Hanoi

Three pegs (Source, Auxiliary, Destination). Move n disks from Source to Destination. Rules: move one disk at a time; never place a larger disk on a smaller one.

The key insight: To move n disks from A to C, you must first move $n-1$ disks to B (using C as aux), then move the largest disk to C, then move $n-1$ disks from B to C (using A as aux).

```

def hanoi(n, source, destination, auxiliary):
    if n == 1:
        print(f'Move disk 1: {source} → {destination}')
        return
    # Step 1: Move top (n-1) disks from source to auxiliary
    hanoi(n-1, source, auxiliary, destination)
    # Step 2: Move the largest disk to destination
    print(f'Move disk {n}: {source} → {destination}')
    # Step 3: Move (n-1) disks from auxiliary to destination
    hanoi(n-1, auxiliary, destination, source)

hanoi(3, 'A', 'C', 'B')
# Output (7 moves):
# Move disk 1: A → C
# Move disk 2: A → B
# Move disk 1: C → B
# Move disk 3: A → C
# Move disk 1: B → A
# Move disk 2: B → C
# Move disk 1: A → C

# Recurrence:  $T(n) = 2 \cdot T(n-1) + 1$ 
# Solving:  $T(n) = 2^n - 1$ 
# n=1: 1 move, n=2: 3, n=3: 7, n=4: 15, n=10: 1023
# Time:  $O(2^n)$   Space:  $O(n)$  stack depth

```

Binary Search — Recursive

Binary search only works on SORTED arrays. It eliminates half the search space each step, giving $O(\log n)$ time.

```
def binary_search(arr, target, low=0, high=None):
    if high is None: high = len(arr) - 1

    # Base case: search space exhausted
    if low > high:
        return -1    # Target not in array

    mid = (low + high) // 2    # Find middle index

    if arr[mid] == target:
        return mid          # Found!
    elif arr[mid] < target:  # Target is in RIGHT half
        return binary_search(arr, target, mid + 1, high)
    else:                   # Target is in LEFT half
        return binary_search(arr, target, low, mid - 1)

# Example trace for arr=[1,3,5,7,9,11,13], target=7:
# low=0, high=6, mid=3 → arr[3]=7 → FOUND at index 3

# Example trace for arr=[1,3,5,7,9,11,13], target=5:
# low=0, high=6, mid=3 → arr[3]=7 > 5 → go left
# low=0, high=2, mid=1 → arr[1]=3 < 5 → go right
# low=2, high=2, mid=2 → arr[2]=5 → FOUND at index 2

# Recurrence:  $T(n) = T(n/2) + O(1)$ 
# Solving (Master Theorem):  $O(\log n)$ 
# On 1,000,000 elements: max 20 comparisons!
# Space:  $O(\log n)$  recursive stack
```

UNIT 1 EXAM TIPS

Q: What is the base case of recursion? → The condition that stops recursion. Without it: StackOverflow.

Q: Why is Fibonacci $O(2^n)$ naive? → Each call makes 2 more calls, tree has 2^n nodes.

Q: How many moves for Hanoi(n)? → $2^n - 1$. For $n=3$: 7. For $n=4$: 15.

Q: Recurrence for Binary Search? → $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$.

Q: Difference between $O(n)$ Factorial and $O(2^n)$ Fibonacci? → Factorial makes 1 recursive call, Fib makes 2.

Q: What does Big O ignore? → Constants and lower-order terms. $O(3n^2 + 5n + 2) = O(n^2)$.

Q: What is amortized complexity? → Average cost per operation over many operations (used in dynamic arrays).

UNIT 2 — Linear Data Structures (10 hrs)

2.1 Arrays

An array stores elements in CONTIGUOUS memory. Each element is accessed by index in $O(1)$ time because the address is: $\text{base_address} + \text{index} \times \text{element_size}$.

1D Array Operations

```
arr = [10, 20, 30, 40, 50]

# Access:  $O(1)$  — direct by index
print(arr[2])          # 30

# Insertion at end:  $O(1)$  amortized
arr.append(60)

# Insertion at position i:  $O(n)$  — must shift elements right
arr.insert(2, 99)       # [10, 20, 99, 30, 40, 50, 60]

# Deletion:  $O(n)$  — must shift elements left
arr.pop(2)              # removes 99

# Search (unsorted):  $O(n)$  — linear scan
# Search (sorted):    $O(\log n)$  — binary search
```

2D Array

```
# 2D Array — matrix representation
matrix = [[1,2,3],[4,5,6],[7,8,9]]

# Access element at row r, col c:  $O(1)$ 
print(matrix[1][2])    # 6

# Traverse entire matrix:  $O(\text{rows} \times \text{cols})$ 
for row in matrix:
    for val in row:
        print(val, end=' ')
    print()
```

Dynamic Array (Python List Under the Hood)

Python's list is a dynamic array. When it runs out of capacity, it doubles its internal buffer (doubling strategy). This makes append $O(1)$ AMORTIZED.

```
class DynamicArray:
    def __init__(self):
        self._data = []          # internal storage
        self._size = 0           # actual elements
        self._capacity = 1       # allocated slots

    def append(self, val):
        if self._size == self._capacity:
            self._resize()        # rare  $O(n)$  operation
        self._data.append(val)    # usually  $O(1)$ 
        self._size += 1

    def _resize(self):
        self._capacity *= 2       # DOUBLING STRATEGY
        new_data = self._data[:]   #  $O(n)$  copy
        self._data = new_data
```

```

        print(f'Resized to capacity {self._capacity}')

    def pop(self):
        if self._size == 0:
            raise IndexError('Array is empty')
        val = self._data[-1]
        self._data = self._data[:-1]
        self._size -= 1
        return val

    def __getitem__(self, idx):
        if idx >= self._size: raise IndexError('out of bounds')
        return self._data[idx]

    def __len__(self): return self._size

# WHY AMORTIZED O(1)?
# n appends total: n/2 at capacity 1, n/4 at cap 2, n/8 at cap 4...
# Total copies = n/2 + n/4 + n/8 + ... = n (geometric series)
# n appends cost n total copies → O(n)/n = O(1) per append

```

2.2 Linked Lists

A linked list is a chain of **NODES**. Each node stores data and a pointer (reference) to the next node. Unlike arrays, nodes are **NOT** in contiguous memory — they're scattered and connected by pointers.

Arrays vs Linked Lists

ACCESS:	Array O(1) by index	vs	Linked List O(n) must traverse from head
INSERT END:	Array O(1) amortized	vs	Linked List O(n) must reach end (unless tail tracked)
INSERT BEG:	Array O(n) must shift	vs	Linked List O(1) just update head pointer
DELETE:	Array O(n) must shift	vs	Linked List O(1) once node found (update pointers)
MEMORY:	Array contiguous	vs	Linked List scattered (extra pointer per node)
BEST FOR:	Arrays: random access		Linked Lists: frequent begin insertions/deletions

Singly Linked List — Complete Implementation

```

class Node:
    def __init__(self, data):
        self.data = data    # stores the value
        self.next = None    # pointer to next node (None if last)

class SinglyLinkedList:
    def __init__(self):
        self.head = None    # points to first node
        self.size = 0

    # — INSERT AT BEGINNING: O(1) —————
    def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head    # new node points to old head
        self.head = new_node        # head updated to new node
        self.size += 1

    # — INSERT AT END: O(n) —————
    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:            # empty list
            self.head = new_node

```



```

        self.size += 1
        return
    current = self.head
    while current.next:          # traverse to last node
        current = current.next
    current.next = new_node      # link last node to new node
    self.size += 1

# — DELETE BY VALUE: O(n) —————
def delete_by_value(self, val):
    if not self.head: return
    if self.head.data == val: # deleting head
        self.head = self.head.next
        self.size -= 1
        return
    current = self.head
    while current.next:        # find node BEFORE the target
        if current.next.data == val:
            current.next = current.next.next # skip over target
            self.size -= 1
            return
        current = current.next
    print(f'{val} not found')

# — TRAVERSE: O(n) —————
def traverse(self):
    current = self.head
    parts = []
    while current:
        parts.append(str(current.data))
        current = current.next
    print(' -> '.join(parts) + ' -> None')

# — SEARCH: O(n) —————
def search(self, val):
    current = self.head
    pos = 0
    while current:
        if current.data == val: return pos
        current = current.next
        pos += 1
    return -1

# Demo:
ll = SinglyLinkedList()
ll.insert_at_end(10)          # 10 -> None
ll.insert_at_end(20)          # 10 -> 20 -> None
ll.insert_at_beginning(5)     # 5 -> 10 -> 20 -> None
ll.traverse()                 # 5 -> 10 -> 20 -> None
ll.delete_by_value(10)        # 5 -> 20 -> None
ll.traverse()

```

Doubly Linked List

Each node has TWO pointers: next (forward) and prev (backward). Allows traversal in both directions and O(1) deletion when you have the node reference.

```

class DNode:
    def __init__(self, data):
        self.data = data
        self.prev = None # pointer to previous node
        self.next = None # pointer to next node

class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

```

```

def insert_at_end(self, data):
    node = DNode(data)
    if not self.head:
        self.head = self.tail = node
        return
    node.prev = self.tail      # new node's prev = old tail
    self.tail.next = node      # old tail's next = new node
    self.tail = node           # tail updated

def insert_after_node(self, existing_node, data):
    if not existing_node: return
    new_node = DNode(data)
    new_node.next = existing_node.next
    new_node.prev = existing_node
    if existing_node.next:
        existing_node.next.prev = new_node
    else:
        self.tail = new_node      # inserted at end
    existing_node.next = new_node

def delete_node(self, node):
    if node.prev: node.prev.next = node.next
    else: self.head = node.next    # deleting head
    if node.next: node.next.prev = node.prev
    else: self.tail = node.prev    # deleting tail

def traverse_forward(self):
    cur = self.head
    while cur: print(cur.data, end=' <-> '); cur = cur.next
    print('None')

def traverse_backward(self):
    cur = self.tail
    while cur: print(cur.data, end=' <-> '); cur = cur.prev
    print('None')

```

2.3 Stack (LIFO — Last In, First Out)

A Stack is like a stack of plates — you can only add or remove from the TOP. Last item added is the first one removed.

Real-world uses: Function call stack, undo/redo, browser back button, expression evaluation, balanced parentheses checking.

```

class Stack:
    def __init__(self):
        self._list = []      # Using Python list as underlying storage

    def push(self, val):      # Add to top — O(1)
        self._list.append(val)

    def pop(self):            # Remove from top — O(1)
        if self.is_empty():
            raise IndexError('Stack is empty')
        return self._list.pop()

    def peek(self):           # View top without removing — O(1)
        if self.is_empty(): return None
        return self._list[-1]

    def is_empty(self): return len(self._list) == 0
    def size(self): return len(self._list)

# — Stack using Linked List —————
class StackLL:

```

```

def __init__(self):
    self.ll = SinglyLinkedList()

def push(self, val):
    self.ll.insert_at_beginning(val) # O(1)

def pop(self):
    if not self.ll.head: raise IndexError('empty')
    val = self.ll.head.data
    self.ll.head = self.ll.head.next
    return val

def peek(self):
    return self.ll.head.data if self.ll.head else None

```

Application: Balanced Parentheses Checker

Use a stack: push every opening bracket. When closing bracket found, check if top of stack matches. At end, stack should be empty.

```

def is_balanced(expression):
    stack = []
    matching = {'(': ')', '[': ']', '{': '}'

    for char in expression:
        if char in '(['{': # opening bracket
            stack.append(char)
        elif char in ')]}': # closing bracket
            if not stack: # nothing to match
                return False
            if stack[-1] != matching[char]: # wrong match
                return False
            stack.pop() # matched, remove from stack

    return len(stack) == 0 # stack must be empty at end

# Trace for '({[]})':
# char='(' → stack=['(']
# char='{ ' → stack=['(', '{']
# char='[' → stack=['(', '{', '[']
# char=']' → top='[' match → pop → stack=['(', '{']
# char='}' → top='{ ' match → pop → stack=['(']
# char=')' → top='(' match → pop → stack=[]
# stack empty → True

print(is_balanced('({[]})')) # True
print(is_balanced('({})')) # False
print(is_balanced('(()))')) # False (unclosed)

```

2.4 Queue (FIFO — First In, First Out)

A Queue is like a line at a counter — first person in is first person out. Elements join at the REAR (enqueue) and leave from the FRONT (dequeue).

Real-world uses: BFS traversal, task scheduling, print queue, CPU process scheduling, breadth-first problems.

```

from collections import deque

class Queue:
    def __init__(self):
        self._dq = deque() # deque for O(1) at both ends

    def enqueue(self, val): # Add to rear — O(1)
        self._dq.append(val)

```

```

def dequeue(self):          # Remove from front - O(1)
    if self.is_empty():
        raise IndexError('Queue is empty')
    return self._dq.popleft()

def front(self):            # View front - O(1)
    if self.is_empty(): return None
    return self._dq[0]

def is_empty(self): return len(self._dq) == 0
def size(self): return len(self._dq)

# — Queue using Linked List —————
class QueueLL:
    def __init__(self):
        self.head = None    # front of queue
        self.tail = None    # rear of queue

    def enqueue(self, val):    # O(1) - add to tail
        node = Node(val)
        if self.tail:
            self.tail.next = node
        else:
            self.head = node    # first element
            self.tail = node

    def dequeue(self):        # O(1) - remove from head
        if not self.head: raise IndexError('empty')
        val = self.head.data
        self.head = self.head.next
        if not self.head: self.tail = None
        return val

    def front(self):
        return self.head.data if self.head else None

```

Stack vs Queue — Quick Reference

STACK (LIFO): push(top), pop(top), peek(top) — Think: pile of plates

QUEUE (FIFO): enqueue(rear), dequeue(front) — Think: queue at shop

Both: O(1) for all core operations when implemented with Linked List

Stack uses: Recursion simulation, undo, bracket checking, DFS

Queue uses: BFS, scheduling, level-order tree traversal

Python: Stack → list.append/pop. Queue → collections.deque with appendleft/popleft

UNIT 3 — Sorting Algorithms: Performance & Trade-offs (10 hrs)

Sorting is the process of arranging data in a specific order (ascending or descending). It's fundamental because many other algorithms (binary search, merging, range queries) require sorted data.

3.1 Sorting Classification

Sorting Algorithm Properties

STABLE: Equal elements maintain their original relative order. (InsertionSort, MergeSort = stable)

UNSTABLE: Equal elements may be reordered. (QuickSort = unstable by default)

IN-PLACE: Sorts using $O(1)$ extra space (only a few variables). (InsertionSort, QuickSort = in-place)

OUT-PLACE: Needs $O(n)$ extra memory. (MergeSort = out-of-place, needs temp array)

COMPARISON: Compares elements pairwise. Lower bound $O(n \log n)$. (Insertion, Merge, Quick)

NON-COMPARISON: Doesn't compare. Can be $O(n)$. (Counting Sort, Radix Sort)

3.2 Insertion Sort

Think of it like sorting playing cards in your hand. Pick each card one by one from right, and insert it into the correct position in the already-sorted left portion.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):      # Start from 2nd element
        key = arr[i]                  # The 'card' we're placing
        j = i - 1
        # Shift elements that are greater than key one position right
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]       # Shift right
            j -= 1
        arr[j + 1] = key               # Place key in correct position
    return arr

# TRACE on [5, 2, 8, 1, 9]:
# i=1: key=2. Shift 5 right. → [2, 5, 8, 1, 9]
# i=2: key=8. 5<8, no shift. → [2, 5, 8, 1, 9]
# i=3: key=1. Shift 8,5,2 right. → [1, 2, 5, 8, 9]
# i=4: key=9. 8<9, no shift. → [1, 2, 5, 8, 9] ✓

# Time: Best  $O(n)$  – already sorted (no shifts needed)
#       Average/Worst  $O(n^2)$  –  $n^2/4$  comparisons on avg
# Space:  $O(1)$  – only variable 'key' extra
# Stable: YES – equal elements not swapped
# In-place: YES
# BEST FOR: Small arrays (<50 elements) or nearly sorted data
```

3.3 Merge Sort

Merge Sort is the classic Divide & Conquer sort. It splits the array in half repeatedly until single elements, then merges them back in sorted order. ALWAYS $O(n \log n)$ regardless of input.

```
def merge_sort(arr):
    if len(arr) <= 1:                # BASE CASE: single element is sorted
        return arr

    # DIVIDE: find midpoint and split
    mid = len(arr) // 2
```

```

    left = merge_sort(arr[:mid])    # recurse on left half
    right = merge_sort(arr[mid:])   # recurse on right half

    # CONQUER: merge sorted halves
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    # Compare front elements of both halves
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:     # <= preserves stability
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    # Append remaining elements
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# TRACE on [38, 27, 43, 3]:
# Split: [38, 27] and [43, 3]
# Split: [38] [27]         [43] [3]
# Merge: [27,38]           [3,43]
# Merge: [3, 27, 38, 43] ✓

# Time: ALWAYS O(n log n) – log n levels, n work per level
# Space: O(n) – needs temp arrays for merging
# Stable: YES
# In-place: NO
# BEST FOR: Large datasets, linked lists, stable sort requirement

```

3.4 Quick Sort

QuickSort picks a PIVOT element, partitions the array so all elements < pivot are left, all > pivot are right, then recursively sorts both sides. Fast in practice but $O(n^2)$ worst case.

```

def quick_sort(arr, low=0, high=None):
    if high is None: high = len(arr) - 1
    if low < high:
        pi = partition(arr, low, high)    # partition index
        quick_sort(arr, low, pi - 1)     # sort left of pivot
        quick_sort(arr, pi + 1, high)    # sort right of pivot
    return arr

def partition(arr, low, high):
    pivot = arr[high]                    # PIVOT = last element (our strategy)
    i = low - 1                          # index of 'smaller than pivot' boundary

    for j in range(low, high):
        if arr[j] <= pivot:              # current element ≤ pivot
            i += 1                        # expand 'smaller' section
            arr[i], arr[j] = arr[j], arr[i] # swap

    # Place pivot in correct position
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1                         # return pivot's final index

# TRACE partition on [3, 6, 8, 10, 1, 2, 1], pivot=arr[6]=1:
# i=-1
# j=0: arr[0]=3 > 1, skip
# j=1: arr[1]=6 > 1, skip
# j=2: arr[2]=8 > 1, skip
# j=3: arr[3]=10 > 1, skip

```

```
# j=4: arr[4]=1 ≤ 1, i=0, swap arr[0]↔arr[4] → [1,6,8,10,3,2,1]
# j=5: arr[5]=2 > 1, skip
# Swap pivot: arr[1]↔arr[6] → [1,1,8,10,3,2,6]
# pivot at index 1

# Time: Best/Avg  $O(n \log n)$ , Worst  $O(n^2)$  – already sorted + last pivot
# Space:  $O(\log n)$  average stack depth
# Stable: NO – pivot swaps can change relative order
# In-place: YES
# BEST FOR: General purpose, cache-friendly, fastest in practice
```

3.5 Performance Measurement

```
import time, random

def measure_time(sort_fn, arr):
    arr_copy = arr[:] # don't modify original
    start = time.perf_counter()
    sort_fn(arr_copy)
    return time.perf_counter() - start

# Generate all 3 types of datasets for each size
for n in [1000, 5000, 10000]:
    rand_arr = [random.randint(0, 100000) for _ in range(n)]
    sorted_arr = list(range(n))
    reverse_arr = list(range(n, 0, -1))

    print(f'\n=== n = {n} ===')
    for label, data in [('Random', rand_arr), ('Sorted', sorted_arr), ('Reverse',
reverse_arr)]:
        t_ins = measure_time(insertion_sort, data)
        t_mrg = measure_time(merge_sort, data)
        t_qck = measure_time(lambda a: quick_sort(a), data)
        print(f'{label:8}: Insertion={t_ins:.4f}s Merge={t_mrg:.4f}s Quick={t_qck:.4f}s')

# EXPECTED FINDINGS:
# InsertionSort: Fast for sorted ( $O(n)$ ), slow for random/reverse ( $O(n^2)$ )
# MergeSort: Consistent  $O(n \log n)$  regardless of input type
# QuickSort: Fast for random, SLOWEST for sorted/reverse (pivot issue)
```

Sorting Comparison Summary

InsertionSort: Simple, in-place, stable. Best for small/nearly-sorted. Worst for large random.

MergeSort: Consistent $O(n \log n)$. Needs extra $O(n)$ memory. Best for large/linked lists.

QuickSort: Fastest average-case. $O(n^2)$ when sorted + last-element pivot. In-place.

Stability matters when: sorting objects by multiple keys (sort by name THEN by age).

Use MergeSort when: stability is required OR guaranteed $O(n \log n)$ is needed.

Use QuickSort when: memory is tight, average performance is key, data is random.

UNIT 4 — Non-Linear & Advanced Data Structures (15 hrs)

4.1 Trees — Fundamentals

A tree is a hierarchical data structure with a root node and subtrees of children, connected by edges. Unlike graphs, trees have no cycles.

Tree Terminology	
Root:	The topmost node (no parent). Only one root per tree.
Leaf:	Node with no children.
Height:	Longest path from root to leaf. Single node tree has height 0.
Depth:	Distance from root to a specific node.
Degree:	Number of children a node has.
Binary Tree:	Each node has at most 2 children (left and right).
BST:	Binary Tree where $\text{left} < \text{root} < \text{right}$ for every node.
Complete BT:	All levels full except possibly the last (filled left to right).
Full BT:	Every node has 0 or 2 children (no node with just 1 child).

Tree Traversals

Three standard ways to visit all nodes in a binary tree:

```
# Given tree:
#           5
#          / \
#         3   7
#        / \ / \
#       1  4 6  9

# INORDER (Left → Root → Right) → gives SORTED output for BST
# Result: 1, 3, 4, 5, 6, 7, 9

# PREORDER (Root → Left → Right) → used for tree copy/serialization
# Result: 5, 3, 1, 4, 7, 6, 9

# POSTORDER (Left → Right → Root) → used for tree deletion
# Result: 1, 4, 3, 6, 9, 7, 5

def inorder(node):
    if node:
        inorder(node.left)
        print(node.val, end=' ')
        inorder(node.right)

def preorder(node):
    if node:
        print(node.val, end=' ')
        preorder(node.left)
        preorder(node.right)

def postorder(node):
    if node:
        postorder(node.left)
        postorder(node.right)
        print(node.val, end=' ')
```


4.2 Binary Search Tree (BST) — Complete Implementation

A BST is a Binary Tree with the BST Property: for every node, all values in its LEFT subtree are SMALLER, and all values in its RIGHT subtree are LARGER.

This property allows $O(\log n)$ search, insert, delete on average (balanced tree). If tree becomes skewed (like a linked list), all operations degrade to $O(n)$.

```
class BSTNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

class BST:
    def __init__(self): self.root = None

    # — INSERT:  $O(\log n)$  avg,  $O(n)$  worst —————
    def insert(self, val):
        self.root = self._insert(self.root, val)

    def _insert(self, node, val):
        if node is None:                # Found insertion point
            return BSTNode(val)
        if val < node.val:               # Go left
            node.left = self._insert(node.left, val)
        elif val > node.val:             # Go right
            node.right = self._insert(node.right, val)
        # val == node.val: duplicate, do nothing
        return node                    # Return unchanged node

    # — SEARCH:  $O(\log n)$  avg —————
    def search(self, val):
        return self._search(self.root, val)

    def _search(self, node, val):
        if node is None: return False   # Not found
        if val == node.val: return True  # Found!
        if val < node.val:               # Must be in left subtree
            return self._search(node.left, val)
        return self._search(node.right, val) # Must be in right

    # — DELETE:  $O(\log n)$  avg - 3 CASES —————
    def delete(self, val):
        self.root = self._delete(self.root, val)

    def _delete(self, node, val):
        if node is None: return None    # Val not in tree

        if val < node.val:               # Go left
            node.left = self._delete(node.left, val)
        elif val > node.val:             # Go right
            node.right = self._delete(node.right, val)
        else:                           # Found node to delete!
            # CASE 1: Leaf node (no children) → just remove
            if not node.left and not node.right:
                return None

            # CASE 2: One child → replace node with its child
            if not node.left: return node.right
            if not node.right: return node.left

            # CASE 3: Two children → find INORDER SUCCESSOR
            # (smallest node in right subtree)
            successor = node.right
            while successor.left:
                successor = successor.left
            # Replace node's value with successor's value
```

```

        node.val = successor.val
        # Delete the successor from right subtree
        node.right = self._delete(node.right, successor.val)

    return node

# — INORDER TRAVERSAL → Sorted Output —————
def inorder(self):
    result = []
    def _inorder(node):
        if node:
            _inorder(node.left)
            result.append(node.val)
            _inorder(node.right)
    _inorder(self.root)
    return result

# Demo:
bst = BST()
for val in [5, 3, 7, 1, 4, 6, 9]:
    bst.insert(val)
print(bst.inorder()) # [1, 3, 4, 5, 6, 7, 9] – sorted!
bst.delete(3)        # Case 3: two children
print(bst.inorder()) # [1, 4, 5, 6, 7, 9]

```

4.3 Heaps

A Heap is a COMPLETE BINARY TREE that satisfies the HEAP PROPERTY. Usually implemented as an array because the complete binary tree structure maps perfectly to array indices.

Heap Properties

MAX-HEAP: Parent $\geq$ Children. Root is always the MAXIMUM element.

MIN-HEAP: Parent $\leq$ Children. Root is always the MINIMUM element.

Complete Binary Tree: All levels full except possibly last (filled left to right).

Array Representation: For node at index i :

→ Left child: index $2*i + 1$

→ Right child: index $2*i + 2$

→ Parent: index $(i-1) // 2$

Use cases: Priority Queue, HeapSort, finding k th largest/smallest element.

```

import heapq

# Python's heapq is a MIN-HEAP
min_heap = []
heapq.heappush(min_heap, 5)
heapq.heappush(min_heap, 2)
heapq.heappush(min_heap, 8)
print(heapq.heappop(min_heap)) # 2 – smallest first

# MAX-HEAP: negate values
max_heap = []
for val in [5, 2, 8, 1]:
    heapq.heappush(max_heap, -val)
print(-heapq.heappop(max_heap)) # 8 – largest first

# HeapSort using heapq:  $O(n \log n)$ 
def heap_sort(arr):
    heapq.heapify(arr) #  $O(n)$  – build heap in-place
    return [heapq.heappop(arr) for _ in range(len(arr))] #  $O(n \log n)$ 

```

4.4 Graphs — Complete Guide

A graph is a set of VERTICES (nodes) connected by EDGES. Unlike trees, graphs can have cycles, multiple paths, and no root.

Graph Types & Terms

DIRECTED (Digraph): Edges have direction. $A \rightarrow B$ means A connects to B but not $B \rightarrow A$.

UNDIRECTED: Edges are bidirectional. $A-B$ means both $A \rightarrow B$ and $B \rightarrow A$.

WEIGHTED: Edges have costs/weights (distances, times).

UNWEIGHTED: All edges treated equally.

CYCLIC: Contains at least one cycle (loop back to a visited node).

ACYCLIC: No cycles. DAG = Directed Acyclic Graph.

VERTEX (V): A node in the graph.

EDGE (E): A connection between two vertices.

ADJACENT: Two vertices are adjacent if there's an edge between them.

DEGREE: Number of edges connected to a vertex.

PATH: Sequence of vertices connected by edges.

CONNECTED: Graph where every vertex can reach every other vertex.

Graph Representations

```
from collections import defaultdict

# — ADJACENCY LIST: space  $O(V+E)$ , best for sparse graphs —
graph_list = defaultdict(list)
# Add directed weighted edges
edges = [(0,1,2), (0,2,5), (1,2,1), (1,3,3), (2,4,4), (3,4,2)]
for u, v, w in edges:
    graph_list[u].append((v, w))

# — ADJACENCY MATRIX: space  $O(V^2)$ , best for dense graphs —
V = 5
matrix = [[0]*V for _ in range(V)]
for u, v, w in edges:
    matrix[u][v] = w      # directed
    # matrix[v][u] = w    # also this for undirected

# Check if edge exists:
# Adj List:  $O(\text{degree of } u)$  — scan u's list
# Adj Matrix:  $O(1)$  — matrix[u][v] != 0
```

BFS — Breadth First Search

BFS visits all neighbors of a node before going deeper. Uses a QUEUE. Finds the SHORTEST PATH in unweighted graphs (fewest edges). Level by level, like spreading water.

```
from collections import deque

def bfs(graph, start):
    visited = set()          # Track visited nodes
    queue = deque([start])   # QUEUE — FIFO
    visited.add(start)
    traversal_order = []

    while queue:
```

```

        node = queue.popleft()    # Process front of queue
        traversal_order.append(node)

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)    # Add to queue (not yet processed)

    return traversal_order

# Time: O(V + E) — every vertex and edge visited once
# Space: O(V) — queue and visited set

# BFS for SHORTEST PATH (unweighted):
def bfs_shortest_path(graph, start, end):
    visited = {start}
    queue = deque([[start]])    # queue of PATHS (not just nodes)
    while queue:
        path = queue.popleft()
        node = path[-1]        # last node in current path
        if node == end:
            return path        # Found shortest path!
        for neighbor, _ in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])    # extend path
    return None    # No path exists

```

DFS — Depth First Search

DFS goes as DEEP as possible down one path before backtracking. Uses RECURSION (or explicit Stack). Useful for finding all connected nodes, detecting cycles, topological sort.

```

def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    result = [start]

    for neighbor, weight in graph[start]:
        if neighbor not in visited:
            result += dfs(graph, neighbor, visited)    # Go deep!

    return result

# DFS up to a certain DEPTH (for friends-of-friends):
def dfs_depth(graph, start, max_depth):
    visited = set()
    result = []
    def _dfs(node, depth):
        if node in visited or depth < 0: return
        visited.add(node)
        if node != start: result.append(node)
        if depth > 0:
            for neighbor, _ in graph[node]:
                _dfs(neighbor, depth - 1)
    _dfs(start, max_depth)
    return result

# Time: O(V + E)  Space: O(V) stack depth

# Demo on our graph:
print('BFS:', bfs(graph_list, 0))    # [0, 1, 2, 3, 4]
print('DFS:', dfs(graph_list, 0))    # [0, 1, 2, 4, 3] (depth first)

```

BFS vs DFS — The Key Difference

BFS: Uses QUEUE. Explores level by level. Finds SHORTEST PATH (unweighted).

DFS: Uses RECURSION/STACK. Goes deep first. Finds ALL paths, detects cycles.

BFS memory: $O(V)$ — can be large (stores entire frontier level).

DFS memory: $O(V)$ — just the current path (usually better in practice).

Use BFS when: finding shortest path, level-order traversal, nearby connections first.

Use DFS when: exploring all possibilities, maze solving, topological sort, cycle detection.

4.5 Hashing — Deep Dive

Hashing maps a KEY to an INDEX using a HASH FUNCTION. Goal: achieve $O(1)$ average time for insert, get, delete.

Hash Function

```
# Simple hash function: modulo operator
def hash_fn(key, table_size):
    return hash(key) % table_size

# Python's built-in hash():
# For strings: complex polynomial hash
# For integers: hash(n) = n (for small n)
# Always modulo table_size to get valid index

# Properties of a GOOD hash function:
# 1. Deterministic: same key → same hash always
# 2. Uniform: distributes keys evenly across table
# 3. Fast:  $O(1)$  to compute
# 4. Minimal collisions
```

Collision Resolution — Separate Chaining

When two keys hash to the same index, a COLLISION occurs. Separate chaining handles this by storing a linked list (or Python list) at each bucket. Multiple key-value pairs can coexist at same index.

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        # Each bucket is a list (chain) - separate chaining
        self.table = [[] for _ in range(self.size)]
        self.count = 0

    def _hash(self, key):
        return hash(key) % self.size    # modulo hash function

    def insert(self, key, value):
        idx = self._hash(key)
        chain = self.table[idx]
        # Check if key already exists → update
        for i, (k, v) in enumerate(chain):
            if k == key:
                chain[i] = (key, value)    # update value
                return
        # Key not found → append new (collision chains here)
        chain.append((key, value))
        self.count += 1

    def get(self, key):
        idx = self._hash(key)
        for k, v in self.table[idx]:
            if k == key:
```

```

        return v
    return None    # key not found

def delete(self, key):
    idx = self._hash(key)
    self.table[idx] = [(k,v) for k,v in self.table[idx] if k != key]
    self.count -= 1

def display(self):
    for i, chain in enumerate(self.table):
        if chain:
            print(f'[{i}]: {chain}')

# Collision Demo (size=5):
ht = HashTable(size=5)
ht.insert('alice', 25)    # hash('alice')%5 might be 2
ht.insert('bob', 30)     # hash('bob')%5 might be 4
ht.insert('carol', 22)   # hash('carol')%5 might be 2 → COLLISION with alice
ht.insert('dave', 28)    # hash('dave')%5 might be 4 → COLLISION with bob
ht.display()
# [2]: [('alice', 25), ('carol', 22)] ← chained!
# [4]: [('bob', 30), ('dave', 28)] ← chained!
print(ht.get('carol'))   # 22 — found by scanning chain
ht.delete('alice')
print(ht.get('alice'))   # None

# Time: O(1) average (short chains), O(n) worst (all in one chain)
# Space: O(n) for n key-value pairs
# Load Factor = n/table_size. Should stay < 0.75 for good performance.

```

4.6 Balanced Trees — Conceptual

Why Balanced Trees?

Problem: BST degrades to $O(n)$ if insertions happen in sorted order (becomes linked list).

Solution: Self-balancing trees automatically restructure to maintain $O(\log n)$ operations.

AVL Tree: Strictly balanced (heights of subtrees differ by max 1). Rotations on insert/delete.

Red-Black Tree: Less strict balance (max 2x height difference). Fewer rotations, used in std libraries.

B-Tree: Multi-way tree for databases & file systems. Millions of records in just 3-4 levels.

Real-world use: Python's dict & set are implemented with hash tables. Java's TreeMap uses Red-Black.

4.7 Tries (Prefix Trees)

A Trie is a tree-like structure for storing strings where each node represents a character. All strings with a common prefix share the same path from the root.

Trie Applications

Autocomplete: Google search suggestions, IDE code completion.

Spell Check: Check if a word exists in dictionary in $O(\text{length of word})$.

IP Routing: Longest prefix matching for network packets.

Word Games: Efficiently find all words with a given prefix.

Search: $O(m)$ per lookup where m = length of word (better than $O(m \log n)$ in BST).

4.8 Important Advanced Structures (Conceptual)

Bloom Filter

A probabilistic data structure for set membership. Uses multiple hash functions + bit array. Can tell you 'DEFINITELY NOT in set' or 'PROBABLY in set' (may have false positives). $O(1)$ insert and lookup. Used in: databases, spell checkers, CDNs.

Spatial Data Structures (K-D Tree, Quadtree)

K-D Tree: Extends BST to k dimensions. Each level splits on a different dimension. Used for nearest neighbor search in geographical apps. $O(\log n)$ queries instead of $O(n)$ brute force.

Quadtree: Divides 2D space into 4 quadrants recursively. Used in image compression, game collision detection, geographic indexing.

Fenwick Tree / Segment Tree

Segment Tree: Stores aggregate (sum, min, max) over ranges. $O(\log n)$ range queries AND updates. Used in competitive programming and database range queries.

Fenwick Tree (BIT): Simpler, memory-efficient for prefix sum queries. $O(\log n)$ update and query.

Skip Lists

Probabilistic alternative to balanced BSTs. Multiple linked lists at different 'speeds' (like express lanes). Expected $O(\log n)$ search. Simpler to implement than AVL trees. Used in Redis.

LAB 5 — Capstone: Simple Social Network Explorer (Full Code)

This capstone combines EVERY concept from the course: Hashing (user profiles), Graphs (friendship network), BFS (shortest path), DFS (friends of friends), Sorting (recommendations), and Trees conceptually.

Complete Social Network Implementation

```
from collections import deque, defaultdict

class SocialNetwork:
    def __init__(self):
        # HashTable (Python dict) → user_id: profile_dict
        self.users = {}
        # Adjacency List → user_id: set of friend_ids (bidirectional)
        self.friends = defaultdict(set)
        # Adjacency List → user_id: set of follower_ids (unidirectional)
        self.followers = defaultdict(set)
        self.following = defaultdict(set)

    # =====
    # USER PROFILE MANAGEMENT (Hashing)
    # =====
    def add_user(self, user_id, name, age, interests):
        if user_id in self.users:
            print(f'[ERROR] User {user_id} already exists!')
            return False
        self.users[user_id] = {
            'name': name,
            'age': age,
            'interests': list(interests) # list of strings
        }
        print(f'[OK] User {name} (ID: {user_id}) added.')
        return True

    def get_user_profile(self, user_id):
        if user_id not in self.users:
            return f'[ERROR] User {user_id} not found.'
        profile = self.users[user_id]
        friends_list = list(self.friends[user_id])
        return (
            f'Name: {profile['name']}\n'
            f'Age: {profile['age']}\n'
            f'Interests: {', '.join(profile['interests'])}\n'
            f'Friends: {friends_list}\n'
            f'Followers: {len(self.followers[user_id])}'
        )

    def update_user_profile(self, user_id, new_details):
        if user_id not in self.users:
            print(f'[ERROR] User {user_id} not found.')
            return
        self.users[user_id].update(new_details)
        print(f'[OK] Profile updated for {user_id}.')

    # =====
    # FRIENDSHIP NETWORK (Graph Operations)
    # =====
    def add_friendship(self, user1_id, user2_id): # Bidirectional
        if user1_id not in self.users or user2_id not in self.users:
            print('[ERROR] One or both users not found.')
            return
        self.friends[user1_id].add(user2_id)
        self.friends[user2_id].add(user1_id)
```



```

        print(f'[OK] {user1_id} and {user2_id} are now friends.')

def add_follow(self, follower_id, followed_id): # Unidirectional
    if follower_id not in self.users or followed_id not in self.users:
        print('[ERROR] User not found.')
        return
    self.followers[followed_id].add(follower_id)
    self.following[follower_id].add(followed_id)
    print(f'[OK] {follower_id} now follows {followed_id}.')

def remove_friendship(self, user1_id, user2_id):
    self.friends[user1_id].discard(user2_id)
    self.friends[user2_id].discard(user1_id)
    print(f'[OK] Friendship removed between {user1_id} and {user2_id}.')

def get_friends_of_user(self, user_id):
    if user_id not in self.users:
        return []
    return list(self.friends[user_id])

# =====
# PATHFINDING - BFS (Shortest Path)
# =====
def find_shortest_path(self, start_id, end_id):
    '''BFS finds SHORTEST path = minimum degrees of separation.'''
    if start_id not in self.users or end_id not in self.users:
        return None
    if start_id == end_id:
        return [start_id]

    visited = {start_id}
    queue = deque([[start_id]]) # Queue stores full PATHS

    while queue:
        path = queue.popleft() # Get next path to explore
        current = path[-1] # Last node in path

        for neighbor in self.friends[current]:
            if neighbor == end_id:
                return path + [neighbor] # FOUND! Return complete path
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])

    return None # No connection found

def degrees_of_separation(self, user1_id, user2_id):
    path = self.find_shortest_path(user1_id, user2_id)
    if path is None: return -1 # not connected
    return len(path) - 1 # edges = nodes - 1

# =====
# DFS - Friends of Friends up to Depth
# =====
def find_friends_of_friends(self, user_id, depth=2):
    '''DFS to find all connections within given depth.'''
    if user_id not in self.users:
        return []

    visited = set()
    result = []

    def _dfs(uid, remaining_depth):
        if uid in visited or remaining_depth < 0:
            return
        visited.add(uid)
        if uid != user_id: # Don't include the start user

```

```

        result.append(uid)
        if remaining_depth > 0: # Still have depth to explore
            for friend_id in self.friends[uid]:
                _dfs(friend_id, remaining_depth - 1)

    _dfs(user_id, depth)
    return result

# =====
# RECOMMENDATION - Hashing + Sorting
# =====
def recommend_friends(self, user_id):
    '''
    DS/Algo approach:
    1. Build interest_map: interest → list of users (Hashing)
    2. Count common interests for each non-friend user (O(interests * users))
    3. Sort by common interest count descending (Sorting)
    4. Return top recommendations (exclude existing friends)
    '''
    if user_id not in self.users:
        return []

    my_interests = set(self.users[user_id]['interests'])
    my_friends = self.friends[user_id]

    # Step 1: Build interest → users hash map
    interest_map = defaultdict(list)
    for uid, profile in self.users.items():
        if uid == user_id: continue
        for interest in profile['interests']:
            interest_map[interest].append(uid)

    # Step 2: Count common interests per potential friend
    common_count = defaultdict(int)
    for interest in my_interests:
        for uid in interest_map[interest]:
            if uid not in my_friends and uid != user_id:
                common_count[uid] += 1

    # Step 3: Sort by count descending
    recommendations = sorted(common_count.items(), key=lambda x: -x[1])

    # Step 4: Return with names
    return [(self.users[uid]['name'], uid, count)
            for uid, count in recommendations]

# =====
# TEXT-BASED MENU INTERFACE
# =====
def run_menu(self):
    while True:
        print('\n' + '='*50)
        print('    SOCIAL NETWORK EXPLORER')
        print('='*50)
        print('1. Add User           2. View Profile')
        print('3. Update Profile      4. Add Friendship')
        print('5. Remove Friendship 6. Follow User')
        print('7. Find Shortest Path      ')
        print('8. Friends of Friends (DFS)')
        print('9. Recommend Friends 10. Exit')
        print('='*50)

        choice = input('Enter choice: ').strip()

        if choice == '1':
            uid = input('User ID: ').strip()
            name = input('Name: ').strip()

```

```

        age = int(input('Age: '))
        intr = [i.strip() for i in input('Interests (comma-separated):
').split(',')]
        self.add_user(uid, name, age, intr)

    elif choice == '2':
        uid = input('User ID: ').strip()
        print(self.get_user_profile(uid))

    elif choice == '3':
        uid = input('User ID: ').strip()
        field = input('Field to update (name/age/interests): ').strip()
        val = input(f'New {field}: ').strip()
        if field == 'age': val = int(val)
        if field == 'interests': val = [i.strip() for i in val.split(',')]
        self.update_user_profile(uid, {field: val})

    elif choice == '4':
        u1 = input('User 1 ID: ').strip()
        u2 = input('User 2 ID: ').strip()
        self.add_friendship(u1, u2)

    elif choice == '5':
        u1 = input('User 1 ID: ').strip()
        u2 = input('User 2 ID: ').strip()
        self.remove_friendship(u1, u2)

    elif choice == '6':
        follower = input('Follower ID: ').strip()
        followed = input('Followed ID: ').strip()
        self.add_follow(follower, followed)

    elif choice == '7':
        u1 = input('Start User ID: ').strip()
        u2 = input('End User ID: ').strip()
        path = self.find_shortest_path(u1, u2)
        if path:
            print(f'Path: {" → ".join(path)}')
            print(f'Degrees of separation: {len(path)-1}')
        else:
            print('No connection found!')

    elif choice == '8':
        uid = input('User ID: ').strip()
        depth = int(input('Depth (e.g. 2): '))
        conns = self.find_friends_of_friends(uid, depth)
        print(f'Connections within {depth} hops: {conns}')

    elif choice == '9':
        uid = input('User ID: ').strip()
        recs = self.recommend_friends(uid)
        print('Recommendations (most common interests first):')
        for name, uid2, count in recs:
            print(f' {name} ({uid2}) - {count} common interests')

    elif choice == '10':
        print('Goodbye!')
        break
    else:
        print('Invalid choice!')

```

```

# =====
# DEMO / TEST SCRIPT
# =====
def demo():
    sn = SocialNetwork()

```

```

# Add users
sn.add_user('alice', 'Alice', 22, ['sports', 'movies', 'music'])
sn.add_user('bob', 'Bob', 25, ['sports', 'tech', 'gaming'])
sn.add_user('carol', 'Carol', 23, ['movies', 'music', 'travel'])
sn.add_user('dave', 'Dave', 28, ['tech', 'sports', 'cooking'])
sn.add_user('eve', 'Eve', 21, ['music', 'movies', 'art'])

# Add friendships
sn.add_friendship('alice', 'bob')
sn.add_friendship('alice', 'carol')
sn.add_friendship('bob', 'dave')
sn.add_friendship('carol', 'eve')

# View profile
print(sn.get_user_profile('alice'))

# Shortest path
print('Path alice→dave:', sn.find_shortest_path('alice', 'dave'))
print('Path alice→eve:', sn.find_shortest_path('alice', 'eve'))
print('Degrees alice→eve:', sn.degrees_of_separation('alice', 'eve'))

# Friends of friends
print('Alice connections (depth 2):', sn.find_friends_of_friends('alice', 2))

# Recommendations
print('Recommendations for alice:', sn.recommend_friends('alice'))

if __name__ == '__main__':
    demo()
    # sn = SocialNetwork()
    # sn.run_menu() # Uncomment for interactive mode

```

Bonus: Geographical Proximity — Which Data Structure?

Question: Find all users within 50km radius. User profiles include (lat, lng) coordinates.

Brute Force: $O(n)$ — check every user's distance. Unusable for millions of users.

K-D Tree: $O(\log n)$ queries. Partitions 2D space by alternating lat/lng splits.

→ Level 0: split on latitude (left = lat < median, right = lat > median)

→ Level 1: split on longitude

→ Level 2: split on latitude again... and so on

Quadtree: Divides map into 4 quadrants recursively. Only drill into nearby quadrants.

Answer: K-D Tree (from Unit 4) is MOST SUITABLE for 2D nearest-neighbor geographical queries.

Why better than BST? BST handles 1D data. K-D handles k-dimensional data.

EXAM PREPARATION — Viva + Coding Questions by Topic

Unit 1 — Recursion & Complexity

TOP VIVA QUESTIONS

Q1: What is the base case in recursion and why is it essential?

A1: Condition that stops the recursion. Without it, function calls itself infinitely → StackOverflow.

Q2: What is the time complexity of recursive Fibonacci? Why?

A2: $O(2^n)$. Each call makes 2 recursive calls → call tree has 2^n nodes.

Q3: How many moves does Tower of Hanoi need for n disks?

A3: $2^n - 1$ moves. For $n=3$: 7. For $n=10$: 1023.

Q4: What is the recurrence relation for Binary Search?

A4: $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$ by Master Theorem.

Q5: What is the difference between $O(n)$ and $O(2^n)$?

A5: For $n=30$: $O(n)=30$ operations, $O(2^n)=1$ billion operations. Exponential is catastrophically slow.

Q6: What is amortized complexity? Give an example.

A6: Average cost per operation over many operations. Dynamic array append: most $O(1)$, occasional $O(n)$ resize, but average is $O(1)$.

Q7: What is space complexity? What's the space complexity of recursive factorial?

A7: Extra memory used. Recursive factorial: $O(n)$ — n stack frames on call stack.

Unit 2 — Linear Data Structures

TOP VIVA QUESTIONS

Q1: Why is insert_at_beginning $O(1)$ for linked list but $O(n)$ for array?

A1: Linked list just updates head pointer. Array must shift ALL elements right.

Q2: What is the difference between Stack (LIFO) and Queue (FIFO)?

A2: Stack: last in, first out (plates). Queue: first in, first out (line at counter).

Q3: How does balanced parentheses checking use a Stack? Trace ' $\{[()]\}$ '.

A3: Push opening brackets. On closing bracket, check top matches. After ' $\{[()]\}$ ': all matched, empty stack → balanced.

Q4: What is the time complexity of all Stack/Queue operations?

A4: All $O(1)$ — push, pop, peek, enqueue, dequeue, front.

Q5: What is the difference between Singly and Doubly Linked List?

A5: Singly: one pointer (next). Doubly: two pointers (prev + next). Doubly allows backward traversal and $O(1)$ deletion with node reference.

Q6: What is amortized $O(1)$ for dynamic array append?

A6: Doubling strategy means resizes happen rarely. Total work for n appends = $O(n)$. Per append average = $O(1)$.

Unit 3 — Sorting

TOP VIVA QUESTIONS

Q1: What is a stable sort? Which of InsertionSort, MergeSort, QuickSort are stable?

A1: Stable sort preserves relative order of equal elements. InsertionSort & MergeSort = stable. QuickSort = unstable.

Q2: When does QuickSort degrade to $O(n^2)$? How to fix it?

A2: When pivot is always min/max (e.g., sorted array + last-element pivot). Fix: random pivot or median-of-three.

Q3: Why is MergeSort always $O(n \log n)$ but uses $O(n)$ space?

A3: Always divides evenly $\rightarrow \log n$ levels, n work per level = $O(n \log n)$. Needs temp arrays for merging $\rightarrow O(n)$ space.

Q4: What does 'in-place' mean? Which sorts are in-place?

A4: Uses $O(1)$ extra memory. InsertionSort and QuickSort are in-place. MergeSort is NOT.

Q5: For 10,000 random integers, which sort would be fastest and why?

A5: QuickSort in practice — $O(n \log n)$ average and cache-friendly. MergeSort also $O(n \log n)$ but needs extra memory.

Q6: What is Divide & Conquer? Which sorts use it?

A6: Split problem, solve parts, combine results. MergeSort and QuickSort both use D&C.

Unit 4 — Non-Linear Structures

TOP VIVA QUESTIONS

Q1: What is the BST property?

A1: For every node: all values in left subtree $<$ node $<$ all values in right subtree.

Q2: What does inorder traversal of a BST give?

A2: Elements in SORTED (ascending) order. Always.

Q3: What are the 3 cases in BST delete?

A3: Case 1: Leaf node → just remove. Case 2: One child → replace with child. Case 3: Two children → replace with inorder successor.

Q4: When does BST become $O(n)$? What is the solution?

A4: When sorted data is inserted → skewed tree (linked list shape). Solution: AVL Tree or Red-Black Tree (self-balancing).

Q5: What is BFS used for vs DFS?

A5: BFS → shortest path (unweighted), level-order traversal. DFS → all paths, cycle detection, topological sort.

Q6: What is a hash collision and how does separate chaining handle it?

A6: Two keys hash to same index. Chaining stores a list at each bucket — multiple pairs at same index.

Q7: What is $O(1)$ for hash table? When does it become $O(n)$?

A7: $O(1)$ when load factor is low (keys spread evenly). $O(n)$ worst case when all keys collide into one bucket.

Q8: For geographical proximity search, which data structure is best?

A8: K-D Tree. Extends BST to k dimensions, $O(\log n)$ nearest-neighbor queries vs $O(n)$ brute force.

Lab 5 — Capstone Social Network

TOP VIVA QUESTIONS

Q1: Why use a Python dict (HashTable) for user profiles?

A1: $O(1)$ average lookup, insert, delete by user_id. Perfect for key-value storage.

Q2: What is the difference between friendship and follow in your implementation?

A2: Friendship: bidirectional (both users added to each other's friend set). Follow: unidirectional (Twitter-like).

Q3: Why does BFS find the shortest path and not DFS?

A3: BFS explores level by level — it reaches the destination via the fewest hops. DFS might take a longer path.

Q4: What does 'degrees of separation' mean?

A4: Minimum number of friendship hops between two users. Alice → Bob → Carol = 2 degrees.

Q5: How does your recommendation system use hashing?

A5: Build interest_map (hash map: interest → users). Scan my interests, find all users with same interests, count overlaps, sort by count.

Q6: What is the overall time complexity of your recommendation function?

A6: $O(I \times U)$ where I = number of interests, U = number of users. Could be optimized with inverted index.

Q7: How does DFS find friends of friends? What is the depth parameter?

A7: DFS from user, decrement depth each hop. Stop when depth=0 or visited. depth=2 means friends AND friends-of-friends.

Coding Questions — Practice These!

MUST-KNOW CODING QUESTIONS FOR EXAM

1. Write recursive factorial. What is time and space complexity?
2. Write recursive Fibonacci. Why is it $O(2^n)$? Write the memoized version.
3. Trace Tower of Hanoi for $n=3$. Show all 7 moves.
4. Write recursive binary search. Trace on $[1,3,5,7,9]$ searching for 5.
5. Implement SinglyLinkedList with insert_at_beginning, insert_at_end, delete_by_value, traverse.
6. Implement Stack using LinkedList with push, pop, peek.
7. Implement Queue using LinkedList with enqueue, dequeue, front.
8. Write is_balanced() using Stack. Trace on '{([[]])}'.
9. Implement InsertionSort. Trace on $[5,2,8,1,9]$.
10. Implement MergeSort with the merge() helper function.
11. Implement QuickSort with partition(). Trace partition step.
12. Write a timer function and measure all 3 sorts on random/sorted/reverse arrays.
13. Implement BST with insert, search, delete (all 3 cases), inorder traversal.
14. Build a directed weighted graph (5-7 nodes). Run BFS and DFS from node 0.
15. Implement HashTable with separate chaining. Show collision example.
16. Implement SocialNetwork: add_user, add_friendship, find_shortest_path (BFS).
17. Implement find_friends_of_friends using DFS with depth parameter.
18. Implement recommend_friends using hash map on interests + sorting.
19. Write the text-based menu interface for SocialNetwork.
20. Explain with code: how to build the social network and test all features.

Ultimate Quick-Reference Mnemonics

- BFS = Queue (FIFO). Think: WATER SPREADING — layer by layer outward.
- DFS = Stack/Recursion. Think: MAZE SOLVER — go as deep as possible, backtrack.
- BST Inorder = SORTED OUTPUT. Always. No exception.
- MergeSort: ALWAYS $O(n \log n)$. Never better, never worse. Needs $O(n)$ extra space.
- QuickSort: FAST on random, TERRIBLE on sorted (with last-pivot strategy). In-place.
- InsertionSort: BEST on nearly-sorted ($O(n)$). Simple. Good for small arrays.
- Hash Table: $O(1)$ average. Magic comes from hash function + chaining handles collisions.
- Stack: LIFO. Plates. push/pop from top.
- Queue: FIFO. Counter line. enqueue rear, dequeue front.

- BST Delete Case 3: Find INORDER SUCCESSOR (smallest in right subtree), copy value, delete successor.
- Amortized $O(1)$ for append: Doubling means total copies = n for n appends $\rightarrow O(1)$ per append on average.
- Tower of Hanoi: $2^n - 1$ moves. $n=3 \rightarrow 7$, $n=4 \rightarrow 15$, $n=10 \rightarrow 1023$.
- Fibonacci naive $O(2^n)$: same subproblems computed exponentially. Memo $\rightarrow O(n)$.
- K-D Tree for 2D: extends BST across 2 dimensions $\rightarrow O(\log n)$ geographic range queries.